Resumo Sistemas Operacionais (Parte Teórica)

Primeira Etapa

Aula 02

1. **O que é um sistema operacional?**

É a camada de software que fica entre os programas e o hardware do computador. Ele roda em modo núcleo, ou seja, possui acesso privilegiado ao hardware e pode executar instruções que os programas comuns não podem executar diretamente.

1.1 Modo Usuário

É o modo em que executam os programas que usamos normalmente, como: navegador, editor, compilador, programa do aluno, interpretador de comandos.

Esses programas possuem acesso restrito ao sistema e não podem acessar diretamente e de forma irrestrita os recursos do hardware. Essa restrição existe principalmente por proteção e segurança. Se qualquer programa pudesse acessar livremente a memória, o disco ou outros dispositivos, um programa poderia interferir no funcionamento de outros programas ou até do próprio sistema operacional.

1.2 Modo Núcleo

É o modo privilegiado de execução. É nesse modo que o kernel (núcleo) do sistema operacional executa. Ele possui privilégios maiores, podendo acessar diretamente os recursos do hardware e executar instruções privilegiadas. Assim o SO consegue controlar recursos como: CPU, memória, disco, dispositivos de entrada e saída.

1.3 Como um programa utiliza os recursos do computador

Um programa que está em modo usuário não acessa diretamente o hardware quando precisa de um serviço por meio do sistema operacional. Ele faz uma solicitação por meio de uma chamada de sistema (system call).

O funcionamento pode ser resumido assim:

Programa em modo usuário → chamada de sistema → Sistema Operacional em modo núcleo → acesso/controla o recurso → retorno ao programa

Por exemplo: se um programa precisa ler um arquivo do disco, ele solicita essa operação ao Sistema Operacional. O SO, executando em modo núcleo, realiza a operação e depois retorna o resultado ao programa.

1.4 Por que existem dois modos?

A separação entre modo usuário e modo núcleo existe principalmente para garantir proteção e segurança.

- ***Modo usuário:*** é quem pede o serviço, tem acesso restrito e é usado pelos programas comuns.
- ***Modo núcleo:*** é quem realiza e controla o serviço, tem acesso privilegiado e é usado pelo kernel do sistema operacional.

1.5 So como máquina estendida

O hardware é complicado e específico para cada dispositivo. Para acessar um disco, por exemplo, seria necessário enviar vários comandos e informar detalhes técnicos, como setor, cilindro, cabeça, etc.

O SO funciona como uma camada de abstração, escondendo os detalhes complicados do hardware e oferecendo recursos mais fáceis de usar, como arquivos.

A abstração esconde os detalhes do hardware, assim, em vez de o programa precisar saber como o disco funciona internamente, ele só precisa pedir para abrir o arquivo, ler, escrever e fechar.

1.6 SO como gerenciador de recursos

Vários programas podem querer usar um mesmo recurso simultaneamente. O SO organiza e controla esse uso, decidindo quem usa o recurso e como ele será utilizado. Existem duas formas principais:

- ***Multiplexação no tempo:*** Os programas se revezam no uso do recurso. Ou seja, um usa por um período e depois outro usa. “Quem usa agora?”
    - Exemplos:
    - CPU: vários processos se revezam usando o processador.
    - Impressora: os trabalhos de impressão precisam ser organizados em uma ordem.

- ***Multiplexação no espaço:*** O recurso é dividido em partes, e cada programa/processo fica com uma parte. “Quem fica com qual parte?”
    - Exemplos:
    - Memória: cada processo recebe uma região de memória.
    - Disco: os dados são armazenados em diferentes partes/blocos do disco.

2. Revisão sobre Hardware de Computadores

O sistema operacional está diretamente ligado ao hardware que ele gerencia. Por isso é necessário conhecer alguns conceitos básicos da máquina.

2.1 Processador (CPU)

A CPU trabalha seguindo um ciclo básico

Buscar → Decodificar → Executar

- Buscar: procura a próxima instrução.
- Decodificar: entende o que a instrução significa
- Executar: realiza a operação.

Como a memória é relativamente lenta, a CPU possui registradores internos, que são pequenas áreas de armazenamento muito rápidas.

- PC(Program Counter): guarda o endereço da próxima instrução que será executada.
- SP (Stack Pointer): aponta para o topo da pilha de execução.
- PSW(Program Status Word): guarda informações sobre o estado atual do programa/processador
    - Dentro do PSW existe um bit de modo: é ele que diz se a CPU está em modo núcleo ou em modo usuário.

Por que a CPU tem vários núcleos?

Uma forma de aumentar o desempenho foi colocar mais capacidade de processamento no mesmo chip, em vez de simplesmente aumentar a frequência da CPU.

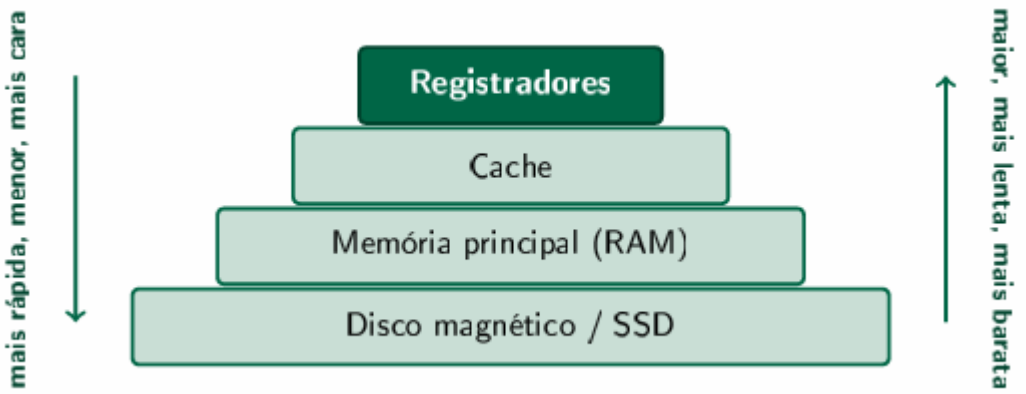
- **Pipeline**: enquanto uma instrução está sendo executada, outras podem estar sendo buscadas ou codificadas.
- **Superescalar**: utiliza várias unidades de execução para realizar operações em paralelo.
- **Multithreading**: um núcleo mantém o estado de mais uma thread e pode alternar entre elas rapidamente.
- **Múltiplos núcleos:** vários processadores completos ficam no mesmo chip.

E por que isso interessa ao SO?

Porque o SO precisa decidir qual processo será executado e em qual núcleo. Além disso, quando vários núcleos trabalham ao mesmo tempo, o SO precisa lidar com situações em que processos podem tentar acessar ou modificar os mesmos dados simultaneamente.

2.2 Memória

Existe uma relação entre velocidade, tamanho e custo da memória. Quanto mais rápida, geralmente menor e mais cara. Quanto maior, geralmente mais lenta e mais barata.



O **Sistema Operacional gerencia essa hierarquia**, decidindo:

- o que deve ficar na RAM;
- o que pode ficar no disco;
- quando trazer dados do disco para a RAM.

O So administra a memória para que os programas tenham acesso aos dados de que precisam, aproveitando os diferentes níveis da hierarquia.

2.3 Discos
O disco possui uma organização física que é mais complexa do que a visão de “arquivo” que o programador utiliza.

Alguns conceitos:

- **Trilha:** uma posição/circunferência onde os dados podem ser armazenados.
- **Cilindro:** conjunto das trilhas que estão na mesma posição nos diferentes pratos.
- **Setor:** unidade que pode ser lida ou escrita.

Tempo de acesso: para acessar uma determinada informação, existem alguns tempos envolvidos:

- Tempo de Busca: tempo para mover o braço até o cilindro correto.
- Latência rotacional: tempo esperando o setor desejado passar pela cabeça de leitura.

Em resumo:

- **O SO possui duas funções principais:**
  - Máquina estendida:Esconde a complexidade do hardware e oferece uma interface mais simples.
  - Gerenciador de recursos:Controla e organiza o uso dos recursos pelos programas.
- **Multiplexação**:
  - No tempo: os programas se revezam no uso do recurso.
  - No espaço: o recurso é dividido entre os programas.

- Multiprogramação:Vários programas ficam na memória e, quando um precisa esperar E/S, outro pode usar a CPU.Isso exigiu:proteção de memória, escalonamento.
- CPU e modo de execução:A CPU possui um bit de modo, relacionado ao PSW, que diferencia:modo usuário × modo núcleo
- Memória: Existe uma hierarquia de memória: Registradores → Cache → RAM → Disco/SSD. O SO gerencia o que fica em cada nível.
- Disco: O hardware possui uma organização física com: trilhas → cilindros → setores Mas o SO esconde esses detalhes e oferece uma interface mais simples baseada em arquivos.

Aula 03 - slide diretórios e navegação linux

3. Diretório

É uma estrutura usada pelo so para organizar arquivos e outros diretórios. Ele funciona como uma pasta, podendo conter: arquivos e outros diretórios (subdiretórios).

O diretório é um tipo especial de arquivo, cujo conteúdo referencia aos arquivos e subdiretórios que estão dentro dele.

Por exemplo:

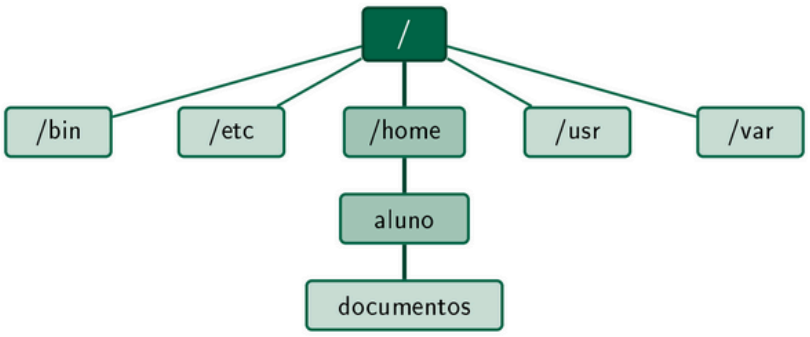
```
/home/aluno/
├── documentos/
│   ├── trabalho.pdf
│   └── aula.txt
├── fotos/
└── programa.c
```

No exemplo, documentos e fotos são diretórios, enquanto trabalho.pdf, aula.txt e programa.c são arquivos.

3.1 Raiz e caminhos

Tudo começa em um único ponto chamado raiz, representado por: **/**

A partir da raiz existem vários diretórios:



O caminho até esse último diretório é `/home/aluno/documentos`

Para chegar ao diretorio documentos: /home/aluno/documentos. Esse é um caminho absoluto porque começa desde a raiz **/**

4. FHS - Filesystem Hierarchy Standard

FHS é um padrão que organiza a estrutura de diretórios dos sistemas Unix/Linux. Ele facilita a organização e a compatibilidade entre diferentes distribuições.

Por exemplo:

```
/
├── bin
├── etc
├── home
├── usr
└── var
```

A idéia é que cada diretório tenha uma função definida, deixando o sistema organizado.

4.1 Diretórios essenciais

- ***/bin:*** contém executáveis essenciais usados pelos usuários, como:
    - ls
    - cp
    - mv
- ***/sbin:*** contém executáveis voltados principalmente para administração e manutenção do sistema, como:
    - mount
    - fdisk
- ***/boot***: contém arquivos necessários para iniciar o sistema, incluindo:
    - kernel
    - GRUB(gerenciador de boot)
- ***/lib:*** contém bibliotecas essenciais utilizadas pelos executavéis de /bin e /sbin, além de componentes relacionados ao kernel

Esses são os diretórios essenciais para a inicialização e funcionamento básico do sistema.

- Em Resumo:
    - ***/boot*** → inicia o sistema
    - ***/bin*** → comandos básicos
    - ***/sbin*** → administração do sistema
    - ***/lib*** → bibliotecas necessárias

4.2 Configuração e Usuários

- ***/etc:*** guarda a maior parte dos arquivos de configuração do sistema e dos serviços. Geralmente são arquivos de texto que definem como o sistema e suas aplicações devem funcionar.
- ***/home:*** contém os diretórios pessoais dos usuários, onde ficam seus arquivos, documentos e configurações individuais.
- ***/root***: é o diretório pessoal do usuário root, o administrador do sistema. Ele fica separado do /home.

- OBS: / e /root são coisas distintas, enquanto / é a raiz de todo o sistema de arquivos, o /root é o diretório pessoal do administrador.

4.3 Dados Temporários e Variáveis

- ***/tmp:*** usado para arquivos temporários criados durante a execução de programas e processos.
    - Em muitos sistemas, esses arquivos podem ser apagados quando o computador é reiniciado. Portanto, não é adequado guardar algo que precisa permanecer após uma reinicialização.
- ***/var:*** guardados que mudam durante o funcionamento do sistema e das aplicações, como:
    - logs
    - caches
    - filas de impressão
    - caixas postais.
- Obs: Se um arquivo é temporário, mas **precisa continuar existindo depois de reiniciar o computador**, ele deve ficar em: /var/tmp

4.4 Dentro de /usr e var

- **/usr**: aplicações e seus arquivos. É onde ficam muitos programas e recursos utilizados pelas aplicações.
    - /usr/bin: executáveis dos programas comuns.
    - /usr/sbin: executáveis administrativos.
    - /usr/lib: bibliotecas utilizadas pelas aplicações.
    - /usr/share: arquivos que não dependem da arquitetura do computador, como documentação e ícones.
- **/var**: dados que mudam com o funcionamento do sistema.
    - /var/log: registros(logs) do sistema.
    - /var/tmp: arquivos temporários que devem permanecer após uma reinicialização.
    - /var/lib: dados e estados mantidos pelas aplicações.
    - /var/spool: filas de variadas, como impressão e e-mail

Em resumo:

- /bin → essencial
- /usr → restante dos programas e recursos
- /var → dados que mudam durante o funcionamento

- E os subdiretórios mais importantes:
- /usr/bin → programas comuns
- /usr/sbin → programas administrativos
- /usr/lib → bibliotecas
- /usr/share → arquivos compartilhados

- /var/log → logs

- /var/tmp → temporários persistentes
- /var/lib → dados das aplicações
- /var/spool → filas

4.5 Dentro de /proc e /dev

Esses dois diretórios são diferentes dos diretórios comuns porque não representam simplesmente arquivos armazenados no disco.

- /proc: funciona como uma interface entre o kernel e o usuário,apresentada na forma de arquivos e diretórios.
    - em /proc tem as informações do kernel e dos processos. Cada processo em execução aparece como um diretório com seu PID (número do processo).
    - Exemplo:

      /proc
      ├── 1
      ├── 250
      ├── 1034
      └── …
      Cada número representa um processo.

- /dev: representa os dispositivos do sistema como arquivos, pode ser encontrado representações de discos, terminais, portas e outros dispositivos.

- obs: processos e dispositivos podem ser acessados usando uma abstração semelhante à usada para arquivos, ou seja, operações como:
  abrir → ler → escrever → fechar

4.6 Visualizando a árvore do sistema

- O comando: /ls mostra o conteúdo do diretório raiz /.

Verá diretórios como: /bin, /boot, /dev, /etc, /home, /lib, /proc, /root, /tmp, /usr, /var

- O comando tree -d -L 1 / mostra a estrutura de diretório em formato de árvore. As opções significam
    - -d: mostra apenas diretórios.
    - -L 1: mostra somente um nível da árvore.
    - /: começa a partir da raiz.
    - então o comando tree -d -L 1 / é basicamente “mostre os diretórios que existem na raiz, sem entrar nos subdiretórios, somente o primeiro nível ”.

5. Caminhos: absoluto x relativo

- Caminho absoluto: começa sempre pela raiz / e funciona independentemente de onde você esteja.
    - /home/aluno/documentos
- Caminho relativo: começa a partir do diretório atual.
    - documentos/projetos

5.1 Descendo e subindo na árvore

- pwd: mostra onde você está
- cd: usado para mudar de diretório.

```
$ pwd
/home/aluno
$ cd documentos
$ pwd
/home/aluno/documentos
```

Também é possível descer vários níveis de uma vez:

```
$ cd projetos/trabalho              # relativo
$ cd /home/aluno/documentos/projetos   # absoluto
```

- cd .. : volta um nível
    - Exemplo:

      /home/aluno/documentos

      ↓

      cd ..

      ↓

      /home/aluno

- cd ../.. : sobe dois níveis

atalhos:

- cd ~: volta para o diretório pessoal do usuário
- cd - volta para o diretório anterior

5.2 diretório atual

O símbolo: . representa o diretório atual. Isso aparece principalmente quando queremos executar um programa que está na pasta atual. Para executar um programa usamos ./

- Por que ./: Normalmente o diretório atual não faz parte do PATH, que é a lista de diretórios onde o sistema procura executavéis.

5.3 Listando arquivos

```
$ ls              # lista o diretorio atual
$ ls -l           # lista detalhada: permissoes, dono, tamanho, data
$ ls -a           # inclui arquivos ocultos (iniciados por ponto)
$ ls -R           # percorre tambem os subdiretorios
$ ls -lh /var/log # combinando opcoes: tamanhos legiveis
```

A opção `-R` (recursiva) aparece em várias ferramentas, sempre com o mesmo sentido: **aplicar a operação também aos subdiretórios**. Exemplo: `cp -R origem/ destino/`

5.4 Criando e removendo diretórios

Diretórios

```
$ mkdir projetos
$ mkdir -p a/b/c
$ rmdir vazio
$ rm -r projetos
```

`rmdir` só remove diretório vazio; `-p` cria a hierarquia inteira.

Arquivos

```
$ touch notas.txt
$ cp notas.txt copia/
$ mv notas.txt prova.txt
$ rm prova.txt
```

`mv` serve tanto para **mover** quanto para **renomear**.

Cuidado

Não existe lixeira no terminal. `rm -r` apaga em definitivo, sem perguntar. Confira o caminho **antes** de pressionar Enter.

- Para diretórios:
    - mkdir: Cria um diretório
    - mkdir -p: criauma hierarquia inteira de diretórios: mkdir -p a/b/c. Mesmo que a e b ainda não existam, o comando cria:

      a/
      └── b/
      └── c/
    - rmdir: Remove um diretório vazio
    - rm -r: remove um diretório e seu conteúdo: rm -r projetos

- Para Arquivos:
    - touch: cria um arquivo vazio
    - cp: copia um arquivo
    - mv: move ou renomeia um arquivo
    - rm: remove um arquivo

5.5 Coringas

Símbolos que permitem selecionar vários arquivos de uma vez:

*	qualquer sequência de caracteres
?	exatamente um caractere
[abc]	qualquer caractere do conjunto

```
$ ls *.txt            # todos os arquivos terminados em .txt
$ ls arquivo?.txt     # arquivo1.txt, arquivo2.txt, ...
$ rm arquivo[12].txt  # remove arquivo1.txt e arquivo2.txt
```

5.6 Encadeando comandos

Ponto e vírgula — executa em sequência

Cada comando roda depois do anterior, mesmo que um deles falhe.

```
$ mkdir teste; cd teste; touch arquivo.txt
```

Pipe — liga a saída de um à entrada do outro

O resultado do primeiro comando vira a entrada do segundo.

```
$ ls -l | grep ".txt"    # so as linhas que contem .txt
$ ls | wc -l             # conta quantos itens existem
$ ls /etc | wc -l        # quantos arquivos de configuracao?
```

5.6 Resumo

Onde as coisas estão

- `/etc` — configuração
- `/home` — usuários
- `/bin`, `/sbin` — essenciais
- `/usr` — aplicações
- `/var` — logs e dados variáveis
- `/tmp` — temporários

Como andar por elas

- `pwd` — onde estou
- `cd` — navegar
- `ls` — listar
- `mkdir` / `rmdir` — criar e remover diretórios
- `touch` / `cp` / `mv` / `rm` — manipular arquivos
- `|` e `;` — combinar comandos

Aula 04

6. MMU - Memory Management Unit

É uma parte do processador responsável por endereços virtuais em endereços físicos da memória. Isso permite que cada processo tenha a ilusão de possuir sua própria memória, mesmo que vários processos estejam usando a RAM ao mesmo tempo.

A MMU, junto com o mecanismo de proteção de memória, impede que um processo acesse diretamente a área de memória pertencente a outro processo.

Exemplo:

Processo A → memória virtual → área física A
Processo B → memória virtual → área física B

O processo A não pode simplesmente acessar a memória física usada pelo processo

O programa funciona assim:

Programa → Endereço virtual →MMU → Endereço físico →RAM

6.1 Custo escondido: chaveamento de contexto

Nessa troca o sistema precisa entre outras coisas: descarregar da cache os blocos modificados e trocar os registros de mapeamento da MMU.

Essas operações têm custo e acontecem muitas vezes por segundo. Por isso, o escalonador não pode trocar de processo o tempo todo, cada troca possui um custo.

7. Dispositivos de E/S: Entre o programa e o dispositivo físico existe uma cadeia:

Programa → Driver →Controlador →Dispositivo

- Driver: é o software específico usado para conversar com o controlador. O driver escreve comandos nos registradores do controlador, lê o estado e o resultado da operação, executa em modo núcleo, pois acessa o hardware diretamente.

- Controlador: é a parte eletrônica que controla o dispositivo e já esconde parte da complexidade do hardware.

7.1 Três maneiras de realizar uma operação de E/S

1. Espera ocupada: O driver inicia a operação e fica consultando o registrador de estado até ela terminar.
   Inicia E/S → fica esperando → termina
   É simples, mas desperdiça a CPU pois ela fica ocupada esperando

2. Interrupção

O driver inicia a operação e libera a CPU. Enquanto o dispositivo trabalha, a CPU pode executar outro processo. Quando termina, o dispositivo avisa a CPU através de uma interrupção.

Inicia E/S →CPU faz outra coisa →Dispositivo termina → Interrupção

3. DMA: no DMA, um mecanismo dedicado transfere os dados sem envolver a CPU durante a transferência.

A CPU é avisada apenas no final.

CPU inicia → DMA transfere os dados → CPU é avisada no final

7.2 Como funciona uma interrupção

Quando um dispositivo termina uma operação:

1. O dispositivo termina;
2. O controlador sinaliza;
3. A CPU interrompe temporariamente o que estava fazendo;
4. Salva o estado atual;
5. Consulta o **vetor de interrupção**;
6. Executa o tratador da interrupção;
7. Retoma o processo.

- Vetor de interrupção: É uma tabela que associa cada número de interrupção ao endereço da rotina responsável por tratá-la. A CPU consulta essa tabela para saber onde deve desviar.
- Em resumo: Interrupção = o dispositivo avisa a CPU quando precisa de atenção.

8. Barramentos

Os componentes do computador precisam de caminhos para trocar dados. Esses caminhos são chamados de barramentos.

Barramento	Uso típico
PCIe	CPU, memória e periféricos
USB	dispositivos externos
SATA	discos rígidos e SSDs

9. Inicialização do computador

A inicialização segue de forma simplificada:

BIOS / UEFI → POST → Setor de boot → Carregador (bootloader) → Kernel

- **BIOS/UEFI:** assume o controle inicialmente.
- **POST:** verifica memória, discos e dispositivos.
- **Setor de boot:** é localizado no dispositivo escolhido para inicialização.
- **Bootloader:** carrega o kernel para a memória.
- **Kernel:** assume o controle, inicializa suas estruturas e inicia os primeiros processos.

Em resumo: **Firmware → bootloader → kernel → processos**

10. Tipos de Sistemas Operacionais

Tipo	Característica principal
Grande porte	Grande volume de E/S; lotes e transações
Servidores	Muitos usuários simultâneos pela rede
Multiprocessadores	Vários processadores na mesma máquina
Computadores pessoais	Interatividade
Portáveis	Energia é um recurso crítico
Embarcados	Função específica e poucos recursos
Nós sensores	Pequenos, sem SO, cooperando em rede
Tempo real	O prazo para executar é requisito
Cartões inteligentes	Poucos KB de memória

10.1 Sistemas de tempo real

- **Tempo real crítico:** perder o prazo significa uma falha
  - Exemplo: freio, controle de voo.
- **Tempo real não crítico:** perder o prazo degrada a qualidade
  - áudio, vídeo

11. Processos

Processo é um programa em execução, junto com seu espaço de endereçamento, seus registradores e sua lista de arquivos abertos.

Programa x Processo

- Programa = código armazenado
- Processo = programa sendo executado

11.1 Tabelas de Processos

É uma estrutura mantida pelo núcleo, contendo uma entrada para cada processo. Ela permite que o sistema: suspenda um processo e depois retome sua execução exatamente de onde parou.

11.2 Árvore de processos

Um processo pode criar outros processos chamados filhos. Esses filhos também podem criar outros.

Processo pai
├── filho
│ └── neto
└── filho

Cada processo recebe um número único chamado PID. Os processos também podem: trocar mensagens, enviar sinais uns aos outros, enviar sinais que solicitam encerramento.

11.3 Quem é o dono do processo?

- UID e GID: Cada usuário possui um identificador chamado UID. Um processo carrega o UID do usuário que o iniciou.

  Um processo filho normalmente herda o UID do processo pai. Os usuários também pertencem a grupos, identificados pelo GID

- Superusuário – root: O root possui um UID especial e ultrapassa praticamente todas as regras de proteção do sistema. Por isso, no uso diário, trabalha-se normalmente com um usuário comum, usando privilégios administrativos apenas quando necessário.
- Em resumo:
  - **UID = identifica o usuário.**
    GID = identifica o grupo.
    PID = identifica o processo.

12. Espaços de endereçamento

Cada processo possui sua própria faixa de endereços. Isso impede que um processo simplesmente alcance a memória pertencente a outro.

E se o programa for maior que a RAM disponível?

A solução apresentada é a memória virtual. O sistema mantém parte do espaço de endereçamento na RAM e parte no disco, movimentando pedaços conforme a necessidade.

Para o programa, continua parecendo que existe um espaço de memória contínuo. A MMU é responsável por sustentar essa ilusão através da tradução dos endereços virtuais.

13. Arquivos, dispositivos e pipes

- Descritor de arquivo: quando um processo abre um arquivo, recebe um número inteiro que passa a representar esse arquivo nas operações seguintes de leitura e escrita. Esse número é chamado de descritor de arquivo.

- Montagem(mount): é o processo de acoplar o sistema de arquivos de outro dispositivo em um ponto da árvore de diretórios existente.

- Arquivos especiais: O sistema representa alguns dispositivos como arquivos. Existem por exemplo:
    - Bloco:discos
    - Caractere: terminais e impressoras.

- Pipe: um pipe é um pseudo arquivo que conecta dois processos.

  Processo A
  │ escreve
  ↓
  PIPE
  ↓ lê
  Processo B

  O que um processo escreve pode ser lido pelo outro. Isso se conecta ao comando que você já viu: ls | wc -l

  O ls produz uma saída e o wc recebe essa saída pelo pipe.

13. Proteção

No UNIX/Linux, os arquivos possuem três conjuntos de permissões:

```
rwx r-x r--
|   |   |
|   |   └── demais usuários
|   └────── grupo
└────────── dono
```

As permissões são:

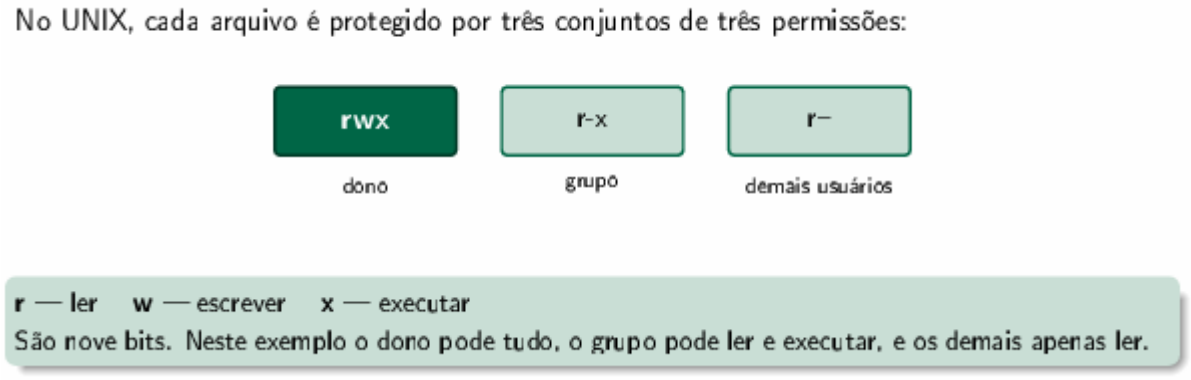
- r(read): ler
- w(write): escrever
- x(execute): executar

São 9 bits de permissão

rwx | r-x | r--
dono | grupo | demais

No exemplo:

- dono → pode ler, escrever e executar;
- grupo → pode ler e executar;
- demais usuários → podem apenas ler.



14. Interpretador de comandos : Shell

O shell não faz parte do núcleo. Ele é um programa comum executado em modo usuário. Sua principal função é:

Ler o que o usuário digita e solicitar ao sistema operacional que execute os comandos.

O shell também, trata:redirecionamento de entrada e saída, pipes, execução de programas em segundo plano. Por isso podem existir vários shells no mesmo sistema operacional.

Na máquina — memória e dispositivos

```
$ cat /proc/self/maps | head -4   # rode duas vezes: mesmos enderecos
$ grep ctxt /proc/stat            # chaveamentos desde o boot
$ vmstat 1 5                      # coluna cs: chaveamentos por segundo

$ lsblk                           # discos e particoes
$ cat /proc/devices               # drivers de bloco e de caractere
$ lsmod | head                    # modulos carregados pelo nucleo

$ cat /proc/interrupts            # o vetor de interrupcao, legivel
                                  # digite algo e rode de novo
```

Dois processos diferentes, os mesmos endereços: é a MMU. E a linha do teclado em `/proc/interrupts` sobe a cada tecla pressionada.

Na máquina — boot, processos e memória virtual

```
$ ls /boot                        # o nucleo e os arquivos do GRUB
$ uname -a                        # qual nucleo esta rodando
$ sudo dmesg | head -25           # o diario do nucleo desde o boot

$ pstree -p | head                # a arvore de processos, com PIDs
$ sleep 300 &                     # cria um filho em segundo plano
$ ps -f                           # veja o PID e o PPID dele
$ kill $!                         # envia o sinal de encerramento

$ free -h                         # RAM e area de troca
$ ps -eo pid,comm,vsz,rss --sort=-vsz | head
```

A soma dos VSZ ultrapassa a memória instalada — porque VSZ é espaço virtual reservado, não RAM ocupada.

Na máquina — arquivos, proteção e shell

```
$ ls -l /proc/self/fd             # descritores 0, 1 e 2
$ ls -l /dev/sda /dev/tty         # 1a letra: b de bloco, c de caractere
$ ls /etc | wc -l                 # um pipe ligando dois processos

$ id                              # seu UID, GID e grupos
$ cp /etc/hostname ~/teste.txt
$ chmod 400 ~/teste.txt           # retira a permissao de escrita
$ echo linha >> ~/teste.txt       # permissao negada
$ chmod 600 ~/teste.txt ; echo linha >> ~/teste.txt

$ ps -p $$                        # o shell e um processo comum
```

Quem recusa a escrita não é o shell: é o núcleo arbitrando o acesso ao recurso. Faça como usuário comum — o root ignora os nove bits.

Slide 05

15. Chamadas de Sistemas

É o mecanismo pelo qual um programa em modo usuário solicita um serviço ao núcleo,

Durante a chamada:

Programa (modo usuário) → chamada de sistema → modo núcleo → SO executa o serviço → modo usuário → programa continua

A CPU muda temporariamente para modo núcleo para executar a operação e depois volta ao modo usuário.

Não é uma chamada de função comum, uma função comum apenas desvia para outro endereço dentro do programa e no mesmo modo de execução.

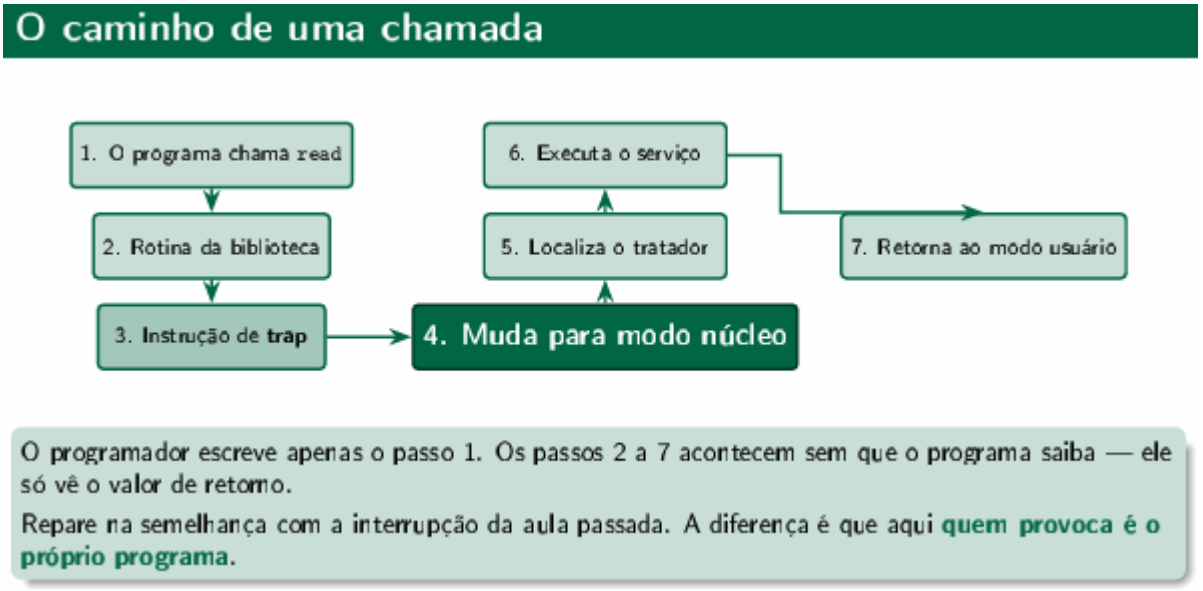
Uma chamada de sistema envolve uma mudança de privilégio. Ela utiliza uma instrução especial chamada trap, que:

- muda o modo de execução
- transfere o controle para o núcleo.

Isso utiliza o bit de modo do PSW.

Em resumo:

- Função comum → muda o fluxo do programa.
- System call → entra no núcleo para solicitar um serviço ao SO.



A principal diferença entre interrupções e chamadas de sistemas é que interrupções normalmente são provocadas por um dispositivo/evento externo e chamadas de sistemas é provocada pelo próprio programa

15.1 Chamadas para gerenciamento de processos

`fork`	cria um processo filho, cópia do pai
`waitpid`	espera um filho terminar
`execve`	substitui a imagem do processo por outro programa
`exit`	encerra o processo e devolve um código de saída

A combinação que constrói tudo

No UNIX, criar processo e executar programa são **operações separadas**. Primeiro o `fork` duplica o processo; depois o filho usa `execve` para virar outro programa.

Parece estranho, mas é o que permite ao pai ajustar o ambiente do filho — redirecionar a saída, montar um pipe — entre uma coisa e outra.

fork + execve

No UNIX, **criar um processo e executar um programa são operações separadas**.

Primeiro:

fork()

Duplica o processo.

Depois, o processo filho pode executar:

execve()

para substituir seu programa pelo outro.

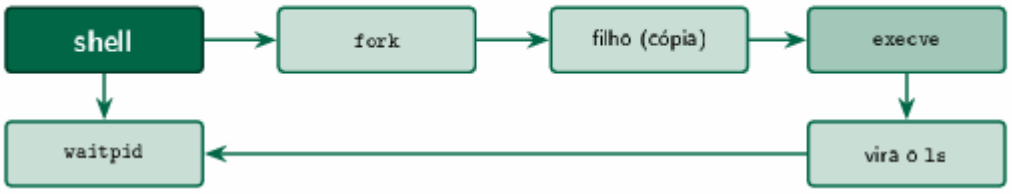
Simplificando:

fork → cria o filho

execve → faz o filho executar outro programa

Essa separação permite que o pai prepare o ambiente do filho antes do `execve`, por exemplo: configurar um pipe;redirecionar a saída;alterar o ambiente.

15.2 Como o shell executa o comando



Toda vez que vocês digitam um comando, é isso que acontece.

E quando vocês põem & no fim do comando? O pai simplesmente não chama `waitpid` — volta a aceitar comandos na hora. É só isso.

O shell não espera o processo terminar com waipid, ele pode voltar imediatamente para aceitar outro comando.

15.3 Arquivos e diretórios

Arquivos

`open`	abre e devolve o descritor
`close`	fecha o descritor
`read`	lê dados do arquivo
`write`	escreve dados
`lseek`	muda a posição de leitura
`stat`	consulta os atributos

Diretórios e sistema de arquivos

`mkdir`	cria diretório
`rmdir`	remove diretório vazio
`link`	cria um novo nome para um arquivo
`unlink`	remove um nome
`mount`	acopla um sistema de arquivos

Reparem que `rm` não existe aqui. O que existe é `unlink`: remove-se o **nome**, e o arquivo desaparece quando o último nome some.

15.4 Outras chamadas

Chamadas diversas

`chdir`	muda o diretório de trabalho
`chmod`	altera as permissões
`kill`	envia um sinal
`time`	consulta a hora do sistema

A API Win32 do Windows

O Windows resolve o mesmo problema com outra interface. Ela tem *milhares* de funções, e muitas delas não são chamadas de sistema: são funções de biblioteca que rodam em modo usuário.

No UNIX a relação entre chamada e função é quase direta; no Windows, não.

A ideia é a mesma nos dois: uma **fronteira controlada** entre o programa e o núcleo. O que muda é o desenho da porta.

16. Estrutura dos Sistemas Operacionais

Na anotação anterior vimos que a chamada de sistema é a passagem controlada entre:
Programa (modo usuário) → syscall / trap → Sistema Operacional (modo núcleo)

A pergunta é: O que existe do outro lado dessa porta? Como o Sistema Operacional é organizado internamente?

A principal questão é: Quanto do sistema deve executar em modo núcleo?

16.1 Muita coisa no modo núcleo

- Vantagem: maior velocidade, pois as partes podem se comunicar diretamente.
- Desvantagens: menor isolamento. Um defeito pode comprometer todo o sistema.

16.2 Pouca coisa no modo núcleo
- Vantagem: maior robustez, pois as partes ficam isoladas.
- Desvantagem: comunicação entre as partes custa mais, pois precisam trocar mensagens.

16.3 Sistemas Monolítos

No sistema **monolítico**, o núcleo é um grande programa executado em modo núcleo. Dentro dele existem, por exemplo:gerenciamento de processos, memória, arquivos, drivers.

As partes podem chamar umas às outras diretamente.

NÚCLEO
├── Processos
├── Memória
├── Arquivos
└── Drivers

- Vantagens: É **rápido**, pois a comunicação interna praticamente não possui o custo de uma troca entre processos.
- Desvantagens: É mais **frágil**.Se uma parte do núcleo tiver um defeito, ela possui privilégios de núcleo e pode comprometer o sistema inteiro.

16.4 Sistemas em camadas
Nesse modelo, o sistema é dividido em **níveis/camadas**. Cada camada utiliza os serviços da camada imediatamente inferior.

Camada superior
↓
Camada abaixo
↓
Camada abaixo
... ↓
Hardware

- Vantagens: Traz mais **organização e disciplina** para o projeto.
- Desvantagens: É difícil manter uma separação completamente rígida e cada passagem entre camadas possui um custo.

16.5 Micronúcleo

No micronúcleo, apenas uma pequena parte permanece no modo núcleo.

```
├── Drivers
├── Arquivos
├── Processos
└── Outros serviços
        ↓ mensagens
Modo núcleo
└── Micronúcleo
```

Grande parte das funções que ficariam dentro do núcleo em um sistema monolítico passa a funcionar como processos em modo usuário.

- Vantagens:Maior robustez e isolamento. Por exemplo, se um driver falhar, ele é apenas um processo que morreu. O sistema pode continuar funcionando e até reiniciar o driver.

- Desvantagens: As partes precisam se comunicar através de mensagens, o que possui um custo maior. O professor cita o MINIX como exemplo de sistema com micronúcleo.
- Micronúcleo = núcleo mínimo + serviços isolados.



No micronúcleo, quase tudo vira **processo de usuário**. Se o driver falhar, ele é só um processo que morreu — o sistema continua de pé e pode reiniciá-lo.

O preço é a comunicação: as partes precisam trocar mensagens, e isso custa. É o modelo **cliente-servidor** aplicado dentro da máquina.

O MINIX, o sistema didático do autor do nosso livro, é um micronúcleo.

16.6 Modelo cliente-servidor

O modelo cliente-servidor pode ser aplicado **dentro da própria máquina**. Um processo solicita um serviço (**cliente**) e outro processo fornece esse serviço (**servidor**).

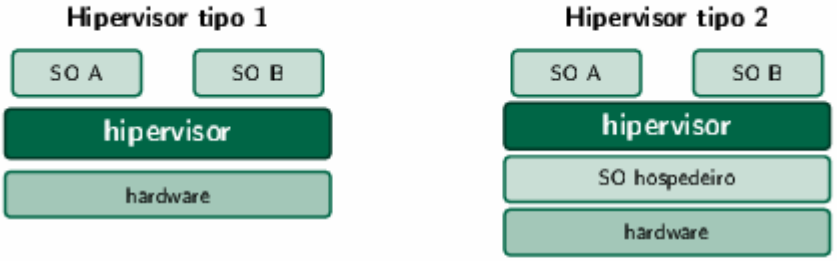
A comunicação ocorre através de mensagens. Isso combina diretamente com a ideia do micronúcleo.

Cliente
↓
mensagem
↓
Servidor

16.7 Máquinas virtuais

O hipervisor é responsável por gerenciar/multiplexar a máquina inteira.

Ele não está apenas multiplexando CPU ou memória: está oferecendo às máquinas virtuais a ideia de que cada uma possui uma máquina completa.



O hipervisor multiplexa a **máquina inteira**, não apenas a CPU ou a memória — multiplexação no tempo e no espaço, agora aplicada ao computador todo.

O VirtualBox de vocês é um hipervisor de tipo 2: roda como programa dentro do Windows do laboratório. Vocês estão dentro do slide.

- Tipo 1 → hipervisor diretamente sobre o hardware.
- Tipo 2 → hipervisor executando sobre um SO hospedeiro.

Exonúcleo

Em vez de dar a cada máquina virtual a ilusão de uma máquina completa, o exonúcleo **particiona** o hardware e entrega pedaços reais a cada uma.

Menos abstração, menos sobrecarga.

Por que C

Quase todo sistema operacional é escrito em C: ela permite acesso direto à memória e ao hardware sem abrir mão de estrutura.

E não usa coleta automática de lixo — o que num núcleo seria inaceitável.

Unidades de medida (seção 1.11)

Na memória, **1 KB** são $2^{10} = 1024$ bytes. Fabricantes de disco usam $10^3 = 1000$. É de onde vem a diferença entre o pendrive de 64 GB anunciado e os 59 GB que aparecem no sistema.

```
$ lsmod | head              # nucleo monolitico, mas modular
$ ls /lib/modules/$(uname -r)/kernel/drivers | head
$ systemd-detect-virt       # em que hipervisor voce esta rodando

$ man 2 open                # secao 2 do manual - chamadas de sistema
$ man 2 syscalls            # a lista completa
```

O que apontar

O `man 2` existe porque chamadas de sistema são uma categoria à parte — funções comuns de biblioteca ficam na seção 3.

E o `systemd-detect-virt` responde `oracle`: é o VirtualBox se identificando. O hipervisor do slide, com nome e sobrenome.

17. Processos e Threads

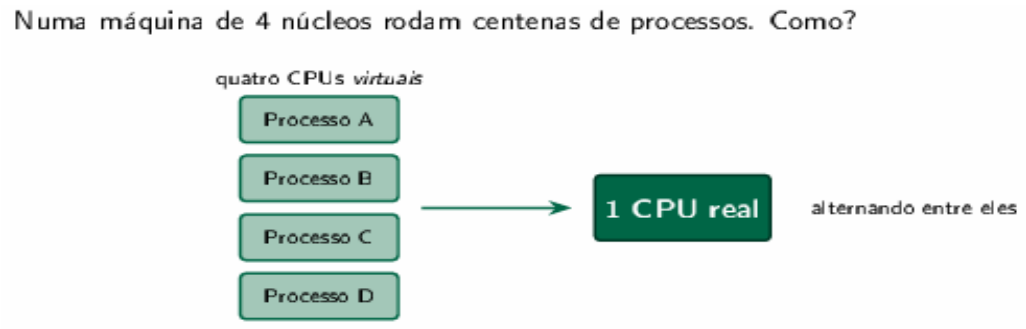
Um **processo** é esse programa **sendo executado**, junto com todas as informações necessárias para que o SO consiga controlar, suspender e retomar sua execução.

Um processo possui, por exemplo:

- espaço de endereçamento;
- registradores;
- contador de programa;
- pilha;
- arquivos abertos;
- estado de execução;
- PID.

17.1 O modelo de processo

Uma máquina pode ter poucos núcleos e, mesmo assim, executar **centenas de processos**. O sistema operacional utiliza o **escalonador** para alternar rapidamente entre os processos.



Pseudoparalelismo

A CPU alterna tão rápido entre os processos que parece haver vários em execução simultânea. Cada processo tem a ilusão de possuir a sua própria CPU — assim como tem a ilusão de possuir a memória inteira.

17.2 Programa não é processo

Uma analogia usada pelo professor:

- **Receita** → programa
- **Ingredientes** → dados de entrada
- **Atividade de preparar o bolo** → processo
- **Cientista** → CPU

A receita continua sendo a mesma, mas a atividade de executá-la pode ser interrompida.

Por exemplo:

1. O cientista começa a fazer o bolo.
2. Ele está seguindo a receita.
3. O filho chega chorando porque foi picado por uma abelha.
4. O cientista interrompe o bolo.
5. Atende o filho.
6. Depois volta para o bolo exatamente de onde parou.

Nesse exemplo, um processo de **prioridade maior** interrompeu temporariamente o outro.

Para guardar: A receita é a mesma. A atividade é outra.

17.3 Como um processo nasce

Quatro eventos criam processos:

1. **Início do sistema** — ao ligar, o núcleo cria os primeiros processos, muitos deles serviços que ficam em segundo plano
2. **Chamada de um processo em execução** — um programa pede ao sistema que crie outro
3. **Solicitação do usuário** — vocês digitam um comando ou clicam num ícone
4. **Início de um serviço em lote** — em sistemas de grande porte, quando chega a vez do trabalho na fila

Em todos os quatro, tecnicamente acontece a mesma coisa: um processo já existente executa uma chamada de sistema para criar outro.

1. Início do sistema

Quando o computador é ligado, o núcleo cria os primeiros processos.Muitos deles são serviços que ficam executando em segundo plano.

2. Um processo em execução cria outro

Um programa pode solicitar ao SO a criação de outro processo. No UNIX, isso é feito através de chamadas de sistema como fork().

3. Solicitação do usuário

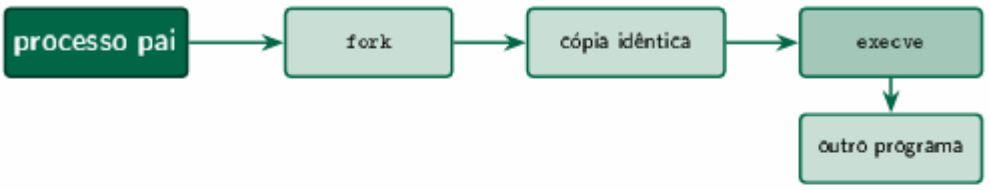
Quando o usuário:

- digita um comando;
- abre um programa;
- clica em um ícone.

O SO precisa criar um processo para executar aquele programa.

4. Início de um serviço em lote

Em sistemas de grande porte, um novo processo pode ser criado quando chega a vez de determinado trabalho na fila.



O que o filho herda

Mesmo espaço de endereçamento (uma cópia), mesmos arquivos abertos, mesmo UID. O que ele **não** herda é o PID: esse é novo.

No Windows, uma única chamada faz as duas coisas de uma vez. São dois projetos diferentes para o mesmo problema.

17.4 Como um processo morre

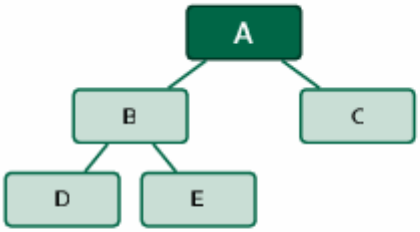
Um processo pode terminar de diferentes maneiras.

- Por vontade própria
    - Saída normal: O processo terminou seu trabalho e chama:exit()
    - Saída por erro: O próprio processo percebe que não consegue continuar e encerra. Exemplos: arquivo não encontrado, argumento inválido.

- Contra a vontade
    - Erro fatal: O processo executa alguma operação inválida. Exemplos:divisão por zero, acesso a uma região de memória proibida.
    - Outro processo solicita seu encerramento: Outro processo pode enviar um **sinal** solicitando o encerramento. Por exemplo: kill PID

Pegadinha importante: kill não significa necessariamente "matar diretamente" o processo. O comando kill envia um sinal para o processo.

17.5 Hierarquia de processos



No UNIX

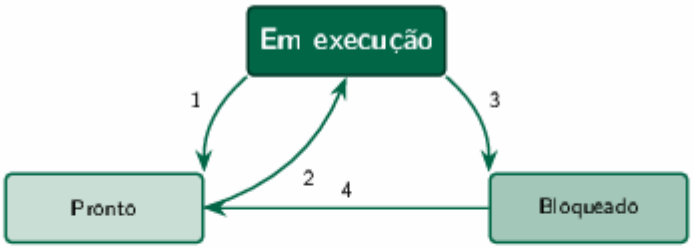
Processos formam uma **árvore**. Pai e filhos permanecem associados num grupo, e um sinal pode ser enviado ao grupo inteiro.

No Windows

Não existe hierarquia. Todos os processos são iguais — o pai recebe apenas um identificador do filho, que pode até passar adiante.

No topo da árvore do Linux está o processo de **PID 1**, criado pelo núcleo no fim do boot. Todos os outros descendem dele.

17.6 Os três estados de um processo



1	o escalonador escolheu outro processo
2	o escalonador escolheu este processo
3	o processo pediu algo e precisa esperar (E/S, por exemplo)
4	o que ele esperava chegou

Não existe transição de bloqueado para em execução. Quem estava esperando volta para a fila.

1. Em execução: O processo está utilizando a CPU naquele momento.

2. Pronto:O processo poderia executar, mas não está executando porque a CPU está ocupada com outro processo. Ele depende do escalonador.

3. Bloqueado: O processo não pode executar naquele momento, mesmo que a CPU esteja disponível. Ele está esperando algum evento externo, como uma operação de E/S.

A diferença entre pronto e bloqueado

Pronto

O processo **poderia** executar agora. Só não está executando porque a CPU está ocupada com outro.

Ele depende apenas do escalonador.

Bloqueado

O processo **não pode** executar, mesmo que a CPU esteja livre. Ele está esperando algo de fora.

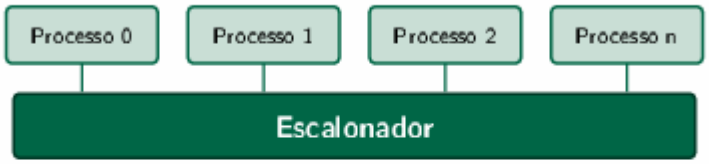
Ele depende de um evento externo.

Exemplo concreto

Vocês digitam `cat arquivo.txt`. O processo pede a leitura do disco e fica **bloqueado**. O disco responde e ele vai para **pronto**. Quando o escalonador escolher, ele passa a **em execução** e imprime na tela.

17.7 O escalonador

O **escalonador** é responsável por decidir qual processo deverá utilizar a CPU. A ideia é que cada processo seja escrito como se fosse o único processo da máquina. O escalonador esconde a complexidade da alternância entre eles.



Pensar assim organiza tudo: existe uma camada de baixo que cuida de interrupções e de decidir quem roda, e **acima dela** existem apenas processos sequenciais.

Cada processo é escrito como se fosse o único na máquina. Toda a complexidade da alternância fica escondida no escalonador.

17.8 A tabela de processos

Como o SO consegue parar um processo e depois continuar exatamente de onde ele estava? O núcleo mantém uma **tabela de processos**.Existe uma entrada para cada processo. Essa entrada é chamada de **Bloco de Controle do Processo (PCB — Process Control Block)**. Ela guarda as informações necessárias para controlar e retomar o processo.

Para poder suspender e retomar, o núcleo mantém uma tabela com uma entrada por processo — o **bloco de controle do processo**.

Gerência de processo	Gerência de memória	Gerência de arquivos
registradores	ponteiro para o código	diretório-raiz
contador de programa	ponteiro para os dados	diretório de trabalho
ponteiro de pilha	ponteiro para a pilha	descritores de arquivo
estado do processo		UID e GID
prioridade		
PID e PID do pai		

Um processo suspenso é sua imagem na memória mais essa linha da tabela.

17.9 O que acontece quando ocorre uma interrupção?

1. Hardware salva informações importantes: O contador de programa e outros registradores são colocados na pilha.
2. Hardware encontra o tratador: O novo contador de programa é carregado a partir do vetor de interrupção.
3. Assembly salva os registradores restantes: Uma rotina inicial salva o restante do contexto.
4. Uma nova pilha é preparada: A rotina monta uma nova pilha para tratar a interrupção.
5. O tratador é executado: O código responsável por tratar a interrupção é executado.
6. O escalonador decide quem executará, Aqui está o ponto mais importante:O processo que volta a executar pode não ser o mesmo que foi interrompido.
7. O contexto do processo escolhido é carregado. Os registradores do processo escolhido são restaurados.

Então:

Interrupção
↓
salva contexto
↓
tratador
↓
escalonador
↓
escolhe processo
↓
restaura contexto
↓
processo escolhido executa

18. Quanto a multiprogramação rende

Se um processo passa a fração **p** do seu tempo esperando E/S, e há **n** processos na memória:

Utilização da CPU

$$\text{utilização} = 1 - p^n$$

Com processos que esperam 80% do tempo ($p = 0{,}8$):

n	1	2	4	6	10
Utilização	20%	36%	59%	74%	89%

Sair de 1 para 2 processos rende 16 pontos. Sair de 6 para 10 rende 15, com o dobro de memória. **O ganho satura.**

O modelo é otimista

Ele supõe que os processos são independentes, o que não é verdade: numa máquina real eles disputam a CPU, o disco e a memória entre si.

Mas a lição vale

Aumentar o número de processos na memória melhora o aproveitamento da CPU — até certo ponto. Depois disso, comprar mais memória para caber mais processos deixa de compensar.

E se a memória acabar, o sistema começa a paginar, e o desempenho **piora**.

Esse "certo ponto" é decisão de projeto — e volta em gerência de memória.

Na máquina — vendo os estados

```
$ sleep 200 &            # um processo que so espera
$ yes > /dev/null &       # um processo que so calcula

$ ps -eo pid,ppid,stat,comm | grep -E "sleep|yes"
```

O que aparece

O `sleep` aparece com **S** — dormindo, ou seja, **bloqueado**. O `yes` aparece com **R** — executando ou pronto.

Os dois estados do diagrama, lado a lado, na mesma tela.

R executando **S** dormindo **D** espera de E/S **T** parado **Z** zumbi

Na máquina — a entrada na tabela de processos

```
$ pgrep -n sleep                    # descubra o PID
$ cat /proc/<PID>/status | head -12 # a entrada na tabela

$ ls /proc/<PID>/fd                 # os descritores de arquivo
$ cat /proc/<PID>/cmdline           # o comando que o originou

$ pkill yes ; pkill sleep           # encerra os dois
```

O que apontar

Os campos State, Pid, PPid e Threads são exatamente as colunas da tabela do slide anterior. Aquele diretório /proc/<PID> é a entrada do processo na tabela, exposta como arquivos. É o /proc da aula de diretórios, agora fazendo sentido completo.

19. Thread

Por que precisamos de threads?

O principal motivo é que muitas aplicações realizam várias atividades ao mesmo tempo e essas atividades precisam acessar os mesmos dados.

Se fossem utilizados processos separados, cada processo teria seu próprio espaço de endereçamento. Compartilhar dados entre eles seria mais trabalhoso e mais caro

19.1 Vantagens das Threads

- São mais baratas: criar uma thread é mais rápido do que criar um processo, porque não é necessário criar um novo espaço de endereçamento.
- Compartilham dados: threads do mesmo processo conseguem acessar os mesmos dados, pois compartilham o mesmo espaço de endereçamento.
- Aproveitam períodos de espera: se uma thread estiver esperando uma operação de E/S, outra thread do mesmo processo pode continuar trabalhando
    - Exemplo:

Thread A → espera E/S
↓
Thread B → continua trabalhando

Em resumo: threads permitem dividir um processo em várias atividades de execução que compartilham os mesmos recursos.

19.2 Um exemplo: editor de texto

Um exemplo concreto: o editor de texto

Imaginem um editor com um documento de 800 páginas. O usuário apaga uma frase na página 1 e pede para ir à página 600.

Com um fluxo só

O editor precisa reformatar o documento inteiro antes de mostrar a página 600.

O usuário fica esperando, com a tela travada.

Com threads

Uma thread atende o teclado. Outra reformata o documento em segundo plano. Uma terceira salva em disco periodicamente.

As três leem e escrevem **o mesmo documento**.

Se fossem três processos, cada um teria a **sua cópia** do documento — e a solução não funcionaria.

19.3 O modelo clássico de thread

Um processo pode possuir várias threads. O processo agrupa os recursos e a thread é a unidade de execução.

As threads do mesmo processo compartilham: espaço de endereçamento, arquivos abertos, variáveis globais e UID. Cada thread possui seu próprio: contador de programa, registradores, pilha e estado.



O processo agrupa os **recursos**. A thread é a unidade de **execução**.

Em resumo:

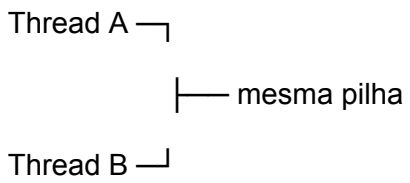
Processo = recursos compartilhados.

Thread = fluxo de execução.

19.4 Por que cada thread precisa da própria pilha?

A pilha guarda informações como: endereços de retorno das funções, e variáveis locais.

Imagine duas threads utilizando a mesma pilha:



Uma poderia sobrescrever as informações utilizadas pela outra. Isso causaria problemas para: retornar das funções, manter variáveis locais, controlar o fluxo de execução.

Por isso, cada thread precisa de sua própria: pilha, contador de programa, conjunto de registradores.

Em resumo: cada thread tem seu próprio fluxo de execução e, por isso, precisa de sua própria pilha, contador de programa e registradores.

Processo x Thread

Processo	Thread
Agrupa recursos	É unidade de execução
Possui espaço de endereçamento próprio	Compartilha espaço com outras threads do processo
Possui arquivos/recursos associados	Usa os recursos compartilhados do processo
Mais caro para criar	Mais barata para criar
Maior isolamento	Maior compartilhamento

19.5 Threads POSIX

No mundo UNIX, o padrão POSIX define uma interface para trabalhar com threads. Essas threads são chamadas de pthreads

O padrão POSIX define a interface de threads usada no mundo UNIX — as **pthreads**.

`pthread_create`	cria uma nova thread
`pthread_exit`	encerra a thread que a chamou
`pthread_join`	espera uma thread específica terminar
`pthread_yield`	libera a CPU para outra thread

Reparem no paralelo: `pthread_create` está para threads assim como `fork` está para processos, e `pthread_join` assim como `waitpid`.
A diferença é que a thread criada **continua no mesmo processo**.

- fork() → cria processo
- pthread_create() → cria thread
- waitpid() → espera processo
- pthread_join() → espera thread

A diferença fundamental é que a thread criada continua dentro do mesmo processo

19.6 Onde as threads são implementadas

No espaço do usuário

O núcleo não sabe que elas existem — enxerga um processo só.

A favor: troca de thread é rapidíssima, sem chamada de sistema.

Contra: se uma thread bloqueia em E/S, **todas param** — o núcleo bloqueou o processo inteiro.

No núcleo

O núcleo mantém uma tabela de threads e escalona cada uma.

A favor: uma thread bloqueada não trava as outras; várias podem rodar em núcleos diferentes.

Contra: cada operação de thread vira chamada de sistema, e isso custa.

O Linux implementa no **núcleo**. É por isso que vocês verão cada thread com o seu identificador.

19.7 Sincronização e condições de corrida

O compartilhamento de dados é uma das principais vantagens das threads. Mas ele também cria um problema: condição de corrida.

19.8 Condição de corrida

Uma condição de corrida ocorre quando duas ou mais threads acessam dados compartilhados e o resultado final depende da ordem em que elas são executadas.

O problema existe porque o escalonador pode interromper uma thread praticamente a qualquer momento, inclusive no meio da operação.

Por isso, um programa pode:

- compilar normalmente
- não apresentar erro de sintaxe
- aparentemente está correto

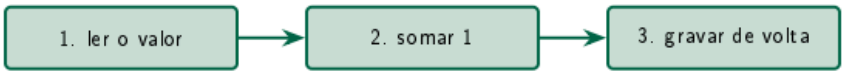
e ainda assim produzir um resultado incorreto. Além disso, o resultado pode mudar de uma execução para outra.

Em resumo: condição de corrida = o resultado depende da ordem de execução das threads.

Duas threads executam o mesmo trecho:

```
contador++;
```

Parece uma operação só. Mas para o processador são **três**:



O escalonador pode interromper a thread entre o passo 1 e o passo 3.

Como um incremento se perde

Momento	Thread A	Thread B	contador
1	lê o valor (10)	—	10
2	*interrompida*	lê o valor (10)	10
3	—	soma 1 → 11	10
4	—	grava 11	**11**
5	soma 1 → 11	—	11
6	grava 11	—	**11**

Duas threads incrementaram. O contador subiu **uma vez só**.

A thread B leu o valor antes de A terminar de gravar. O trabalho de A foi sobrescrito e perdido.

19.9 Região crítica

Uma região crítica é o trecho do programa que acessa ou modifica dados compartilhados e que precisa ser protegido contra acesso simultâneo.

No exemplo:

```
contador++;
```

é a região crítica.

A ideia é:

Thread A → entra na região crítica
↓
executa
↓
sai
Thread B → espera
↓
entra depois

Enquanto uma thread estiver dentro de sua região crítica, outra não deve entrar na região crítica que acessa o mesmo recurso.

20. Quatro condições para uma solução correta

Quatro condições para uma solução correta:

1. Nunca dois processos simultaneamente dentro das regiões críticas
2. Nenhuma suposição sobre velocidade ou número de CPUs
3. Nenhum processo fora da sua região crítica pode bloquear outros
4. Nenhum processo deve esperar eternamente para entrar

Guardem essas quatro. Na próxima aula elas serão o critério para julgar cada solução proposta.

As quatro para decorar:

- Nunca dois dentro
- Não depender da velocidade/CPUs
- Quem está fora não bloqueia
- Ninguém espera para sempre

Essas quatro condições serão utilizadas para avaliar as soluções de sincronização.

Prática 1 — digite o programa `threads1.c`

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

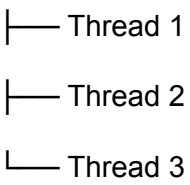
void *tarefa(void *arg) {
    printf("Thread %d  --  PID: %d\n", *(int *) arg, getpid());
    sleep(60);
    return NULL;
}

int main(void) {
    pthread_t t[3];
    int id[3] = {1, 2, 3};

    for (int i = 0; i < 3; i++)
        pthread_create(&t[i], NULL, tarefa, &id[i]);
    for (int i = 0; i < 3; i++)
        pthread_join(t[i], NULL);
    return 0;
}
```

A ideia principal do programa é criar três threads:

Processo



Cada thread imprime seu identificador e o PID do processo.

Para compilar: gcc -o threads1 threads1.c -pthread

⚠️ A opção: -pthread

É obrigatória para compilar esse programa com pthreads.

Executar: ./threads1 &

Ver quantas threads existem: grep Threads /proc/$(pgrep -n threads1)/status

Ver os identificadores das threads: ls /proc/$(pgrep -n threads1)/task

Encerrar: pkill threads1

O que observar? As três threads imprimem o mesmo PID porque pertencem ao mesmo processo. Por outro lado, cada thread possui seu próprio identificador.

Ideia importante: Threads diferentes podem pertencer ao mesmo processo e, portanto, compartilhar o mesmo PID do processo.

Prática 2 — digite o programa `contador.c`

```c
#include <stdio.h>
#include <pthread.h>
#define VOLTAS 10000000
int contador = 0;           /* variavel compartilhada */

void *incrementa(void *arg) {
    for (int i = 0; i < VOLTAS; i++)
        contador++;         /* a regiao critica */
    return NULL;
}

int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, incrementa, NULL);
    pthread_create(&t2, NULL, incrementa, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Esperado: %d\n", 2 * VOLTAS);
    printf("Obtido:   %d\n", contador);
    return 0;
}
```

O programa possui: int contador = 0;

Essa variável é compartilhada pelas duas threads.

Cada uma executa:

for (int i = 0; i < VOLTAS; i++)

contador++;

Se:

VOLTAS = 10.000.000

e existem duas threads, o esperado é: 20.000.000

Mas o resultado pode ser menor e variar entre execuções. Isso ocorre por causa da condição de corrida.

O programa:

- está sintaticamente correto;
- compila;
- executa;

mas está semanticamente errado devido ao acesso concorrente à variável compartilhada.

⭐ Para guardar:

> Um programa pode estar correto em sintaxe e ainda assim estar errado por causa da concorrência.

Soluções que parecem funcionar, mas não funcionam

- Desabilitar interrupções:
    - A ideia seria: desabilitar interrupções → executar região crítica → habilitar interrupções.
    - Se ninguém puder interromper a execução, outra thread não entraria no meio.

- Por que não funciona para programas comuns?

Dar esse poder a um programa de usuário seria perigoso. Se o programa nunca habilitasse novamente as interrupções, poderia comprometer o sistema.
Alem disso, em uma máquina com vários núcleos, desabilitar interrupções em um núcleo não impede outro núcleo de executar.

21. Variável de trava

Outra ideia é usar uma variável indicando se a região está ocupada:

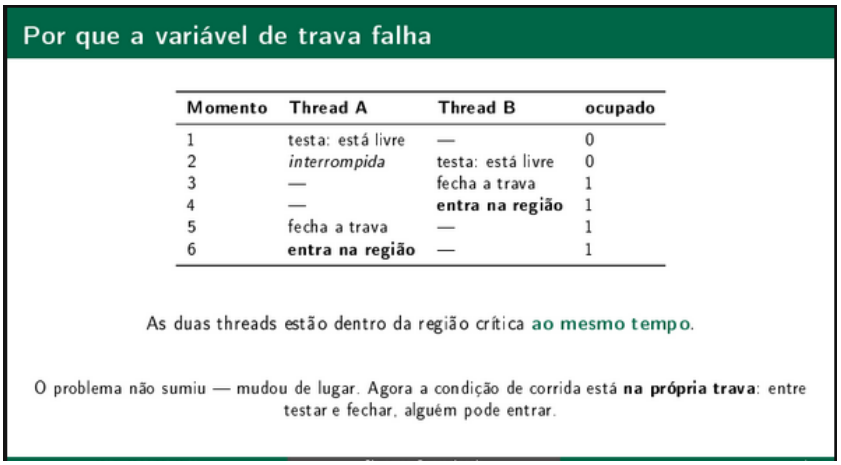
```
while(ocupado);

ocupado = 1;

contador++;

ocupado = 0;
```

parece funcionar, mas existe uma condição de corrida na própria trava



Por que a variável de trava falha

Momento	Thread A	Thread B	ocupado
1	testa: está livre	—	0
2	*interrompida*	testa: está livre	0
3	—	fecha a trava	1
4	—	**entra na região**	1
5	fecha a trava	—	1
6	**entra na região**	—	1

As duas threads estão dentro da região crítica ao mesmo tempo.

O problema não sumiu — mudou de lugar. Agora a condição de corrida está **na própria trava**: entre testar e fechar, alguém pode entrar.

22. Instrução TSL e espera ocupada

O hardware pode fornecer uma instrução especial chamada TSL. Ela consegue testar e fechar a trava em uma única operação indivisível.

Assim, outra thread não consegue entrar entre o teste e o fechamento. Porém surge outro problema: espera ocupada.

A thread fica fazendo:

Está livre? não

Está livre? não

Está livre? não…

Ela continua consumindo CPU sem realizar trabalho útil.

O problema da espera ocupada: A cpu é utilizada para ficar esperando.

Por isso surge uma ideia melhor: Em vez de ficar testando, a thread pode dormir enquanto espera.

23. Semáforos

A ideia do semáforo é: quem não pode entrar fica bloqueado e libera a cpu para outros. Quando chegar a vez, o sistema pode acordar uma das threads que estava esperando.

O semáforo possui duas operações principais:

- Down: se o valor do semáforo for maior que zero, decrementa e segue. Se é zero, o processo dorme.
- Up: incrementa o semáforo, se houver alguém esperando(dormindo), acorda um deles.

Em resumo: semáforo é um mecanismo de sincronização que permite bloquear quem precisa esperar em vez de desperdiçar CPU

24. Mutex

Quando o objetivo é simplesmente proteger uma região crítica, podemos utilizar um mutex.

Mutex vem da ideia de mutual exclusion(exclusão mútua)

Ele possui dois estados: LIVRE, OCUPADO

as operações principais são:

- Lock → entra
- Unlock → sai

Quando o semáforo só precisa proteger uma região crítica, basta uma versão simplificada, com **dois estados**: livre e ocupado.



- `lock` — se o mutex está livre, fecha e entra. Se está ocupado, a thread fica bloqueada até ele abrir
- `unlock` — abre o mutex e libera uma thread que estava esperando

O teste e o fechamento são indivisíveis. É isso que a variável de trava não tinha — e é o que **conserta o nosso contador**.

```c
#include <stdio.h>
#include <pthread.h>
#define VOLTAS 10000000
int contador = 0;
pthread_mutex_t trava = PTHREAD_MUTEX_INITIALIZER;
void *incrementa(void *arg) {
    for (int i = 0; i < VOLTAS; i++) {
        pthread_mutex_lock(&trava);     /* entra na regiao critica */
        contador++;
        pthread_mutex_unlock(&trava);   /* sai da regiao critica */
    }
    return NULL;
}
int main(void) {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, incrementa, NULL);
    pthread_create(&t2, NULL, incrementa, NULL);
    pthread_join(t1, NULL);  pthread_join(t2, NULL);
    printf("Esperado: %d   Obtido: %d\n", 2 * VOLTAS, contador);
    return 0;
}
```

Corrigindo o contador com mutex

Antes: contador++; sem proteção.

Agora:

pthread_mutex_lock(&trava);

contador++;

pthread_mutex_unlock(&trava);

A região crítica fica protegida:

lock
↓
REGIÃO CRÍTICA
contador++
↓
unlock

Somente uma thread pode executar essa parte por vez.

O resultado passa a ser o esperado:

- Esperado: 20000000
- Obtido: 20000000

em todas as execuções corretas do programa. Mas o mutex deixa o programa mais lento Isso é esperado.

Existe um custo para:

- adquirir o mutex;
- liberar o mutex;
- bloquear;
- acordar;
- sincronizar as threads.

Mesmo assim: Um programa mais lento e correto é melhor que um programa rápido que produz resultados errados.

25. Escalonamento

Depois de resolver o problema da sincronização, surge outra pergunta:

Quem decide qual processo ou thread executa agora? Essa é a função do escalonador

- Quando o sistema escalona?

O escalonador precisa tomar decisões principalmente em quatro situações:

1. *O processo é criado*

É preciso decidir: executa o pai ou executa o filho

2. *Processo termina*

A CPU precisa ser entregue para outro processo.

3. *Processo bloqueia*

Por exemplo: E/S, semáforo, mutex. O processo não pode continuar executando, então outro precisa assumir a CPU.

4. *Chega uma interrupção*

Por exemplo: interrupção de E/S, interrupção do relógio. O sistema pode aproveitar o evento para realizar um novo escalonamento.

25. Escalonamento não preemptivo

No escalonamento não preemptivo, o processo continua executando até: terminar, bloquear.

O escalonador não o tira da CPU à força.

processo executa → bloqueia ou termina → escalonador escolhe outro

26.Escalonamento preemptivo

No escalonamento preemptivo,o processo recebe um tempo máximo de execução. Quando esse tempo acaba, o relógio gera uma interrupção.

O sistema pode então:

Processo A → tempo acaba → interrupção → escalonador → processo B

A diferença:

- **Não preemptivo:** o processo sai voluntariamente ao bloquear ou terminar.
- **Preemptivo:** o SO pode interromper o processo e escolher outro.

27. Dois tipos de processo

- *Limitado pela CPU – CPU-bound*

Passa a maior parte do tempo realizando cálculos. Características: usa CPU durante longos períodos, raramente espera E/S.
Exemplo: yes, md5sum.

- *Limitado por E/S – I/O-bound*

Passa grande parte do tempo esperando: disco, rede, teclado, outras operações de E/S.
Utiliza a CPU em períodos curtos.
Exemplo: editor de texto, enquanto espera o usuário digitar.

- Por que processos limitados por E/S podem receber prioridade?

Eles usam a CPU rapidamente e depois voltam a esperar. Isso pode ajudar a manter outros recursos, como disco e rede, funcionando sem ficar esperando desnecessariamente.

- CPU-bound → calcula muito → espera pouco
- E/S I/O-bound → calcula pouco → espera muito E/S

28. O que um bom escalonador busca?

Depende do tipo do sistema.

- *Em todos os sistemas*

busca-se justiça, cumprir a política de escalonamento, manter os recursos ocupados.

- *Sistemas em lote*

Interessa executar muitos trabalhos por hora, reduzir o tempo de retorno.

- *Sistemas interativos*

Interessa responder rapidamente ao usuário.

- *SIstemas de tempo real*

Interessa cumprir os prazos.

Tempo de retorno

É o tempo entre o momento em que um trabalho é submetido e o momento em que ele termina. Esse valor será usado para comparar os algoritmos apresentados.

Importante:
Não existe um algoritmo de escalonamento que seja o melhor para todos os tipos de sistema. O que é interessante para um sistema em lote pode não ser adequado para um sistema interativo.

29. FCFS e SJF
FCFS → ordem de chegada. SJF → menor trabalho primeiro.

Quatro trabalhos chegam juntos. Tempos de execução: **A = 8**, **B = 4**, **C = 2**, **D = 1**.

FCFS — primeiro a chegar	SJF — menor primeiro
Ordem: A, B, C, D	Ordem: D, C, B, A
Terminam em: 8, 12, 14, 15	Terminam em: 1, 3, 7, 15
Tempo médio de retorno: $(8+12+14+15)/4 = \mathbf{12{,}25}$	Tempo médio de retorno: $(1+3+7+15)/4 = \mathbf{6{,}5}$

O mesmo trabalho, na mesma máquina, com metade do tempo médio. **Só mudou a ordem.**

O problema do SJF: ele exige saber de antemão quanto cada trabalho vai durar — o que quase nunca se sabe.

30. Round-robin

O round-robin é muito utilizado como ideia para sistemas interativos. Cada processo recebe uma pequena quantidade de tempo chamada quantum.

Exemplo:

A → quantum → B → quantum → C → quantum → D

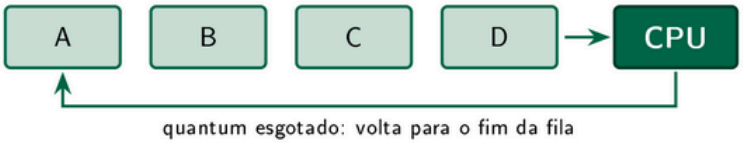
Se o processo não terminou quando seu quantum acaba:

processo → volta para o final da fila

Exemplo:

A B C D
↓
A executa
↓
A não terminou
↓
B C D A

Para sistemas interativos: cada processo recebe uma fatia de tempo, o **quantum**. Se não terminar, vai para o fim da fila.



Quantum curto demais

Muitas trocas de processo. E cada troca é um **chaveamento de contexto**, que custa caro.

Quantum longo demais

O sistema demora a responder. No limite, vira FCFS.

Em resumo:
Round-robin = cada processo recebe uma fatia de tempo e, se não terminar, volta para o fim da fila.

31. Escalonamento por prioridades

Cada processo recebe uma prioridade. O escalonador escolhe o processo pronto de maior prioridade.

Exemplo:

Pronto:

A → prioridade alta
B → prioridade média
C → prioridade baixa
↓
Escalonador escolhe A

- ***Problema inanição***

Pode acontecer de processos de baixa prioridade ficarem esperando indefinidamente. Se sempre houver alguém de prioridade alta, os de prioridade baixa nunca executam.

Imagine:

Alta prioridade → sempre chega alguém
Alta prioridade → executa
Alta prioridade → executa
Alta prioridade → executa
...
Baixa prioridade → nunca executa

Uma forma de evitar isso é aumentar gradualmente a prioridade de quem está esperando há muito tempo.

- ***nice no Linux***

No linux, o valor do nice está relacionado à prioridade de escalonamento.

Vai de −20 (prioridade máxima) a 19 (prioridade mínima).

Um valor nice maior significa que o processo está sendo mais ”gentil” e cede mais CPU aos outros. Usuários comuns podem aumentar a própria nice, mas não podem simplesmente diminuir sua prioridade para valores mais privilegiados.

Prática 2 — dois processos, um núcleo

```
$ taskset -c 0 yes > /dev/null &
$ taskset -c 0 nice -n 19 yes > /dev/null &

$ ps -eo pid,ni,pcpu,comm --sort=-pcpu | head -4

$ pkill yes
```

Anote as respostas

1. Qual a porcentagem de CPU de cada um? Qual a coluna `NI` de cada um?
2. Os dois fazem exatamente a mesma coisa. Por que um recebe muito mais CPU que o outro?
3. O `taskset -c 0` prende os dois no núcleo 0. O que aconteceria sem ele, numa máquina com dois núcleos?

taskset -c 0 yes > /dev/null &

e:

taskset -c 0 nice -n 19 yes > /dev/null &

Os dois processos fazem essencialmente a mesma atividade, mas possuem prioridades diferentes.

- *E se não usarmos tasket?*

Se houver mais de um núcleo disponível e não prendermos os processos ao mesmo núcleo, o SO poderá escaloná-los em núcleos diferentes. Assim, a disputa pela mesma CPU pode diminuir.

taskset -c 0 restringe o processo ao núcleo 0.

Segunda etapa

26.

processo são todos os recursos em execução.

As thread é um bloco de execução

_ _ _ _ _ _ _ _ _ _ _ _ _ _ - - -

varias threads compoe um procesoo

e um processo compoe vários programas